

Arquitectura de Datos

Especialización en Internet de las Cosas

Docente:

Lacheski, Martín Anibal

Clase 6:

Tecnologías Especializadas de Datos



Arquitectura de Datos

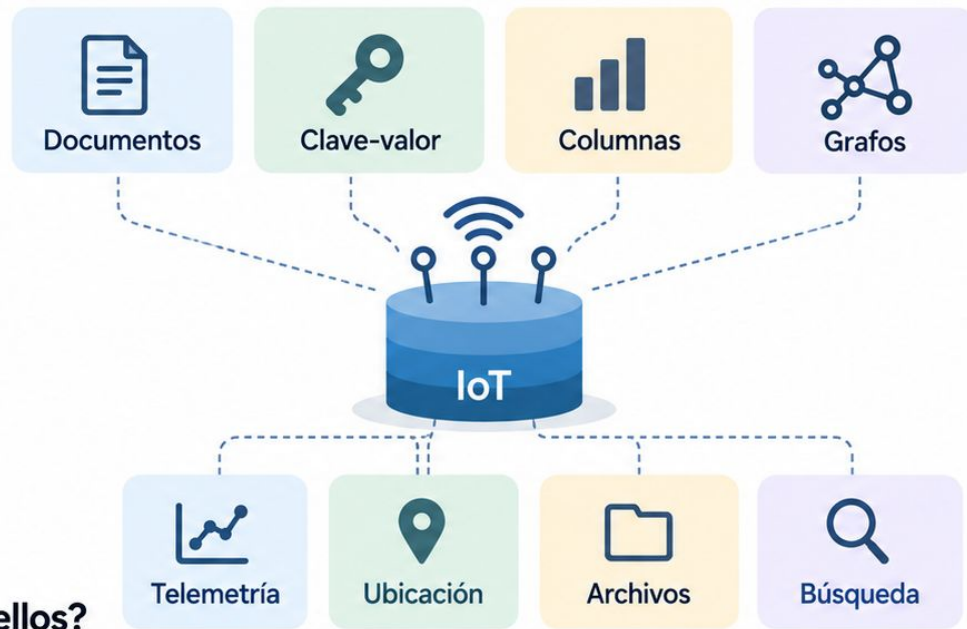
Ya conocemos diferentes modelos de Bases de Datos

En la clase anterior analizamos:

- bases documentales;
- clave-valor;
- bases columnares distribuidas;
- grafos;
- disponibilidad y consistencia;
- CAP;
- persistencia polígota.

Ahora aparece otra pregunta:

¿qué ocurre cuando las características de determinados datos requieren mecanismos especialmente optimizados para trabajar con ellos?



No siempre necesitamos otro modelo completo.
A veces necesitamos una capacidad especializada para un problema concreto.

Hasta ahora elegimos modelos

Hoy vamos a observar características específicas de los datos

Clase 5



Clase 6



Una necesidad especializada no implica reemplazar toda la arquitectura existente.

Objetivos de la clase

Seleccionar almacenamiento según cómo se comportan los datos

Vamos a aprender a:

- reconocer datos de series temporales;
- analizar consultas y agregaciones temporales;
- aplicar retención y downsampling;
- trabajar conceptualmente con información geoespacial;
- comprender caché y almacenamiento temporal;
- separar archivos y metadatos;
- comprender object storage;
- introducir embeddings y búsqueda vectorial;
- comprender cómo funciona una arquitectura RAG;
- diferenciar recuperación y generación;
- utilizar conceptualmente Text-to-SQL;
- combinar diferentes mecanismos de persistencia.



La especialización debe responder a una característica concreta de los datos o de la forma en que necesitamos utilizarlos.

El caso vuelve a crecer

Ahora aparecen datos con necesidades diferentes

El sistema de monitoreo posee ahora:

- cientos o miles de dispositivos;
- mediciones frecuentes;
- información histórica;
- ubicación de dispositivos;
- estado actual consultado constantemente;
- configuraciones variables;
- manuales y documentación técnica;
- fotografías y otros archivos;
- usuarios que quieren consultar información mediante lenguaje natural.

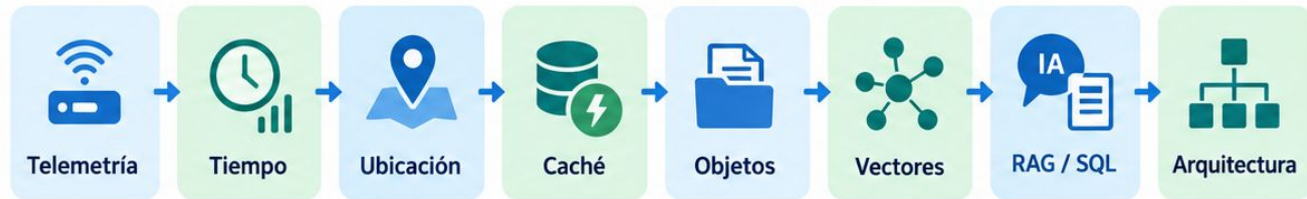


El mismo sistema puede contener datos que necesitan mecanismos de almacenamiento diferentes.

Arquitectura de Datos

Agenda del día

- Características de los datos IoT.
- Series temporales.
- Agregaciones temporales.
- Retención y downsampling.
- Datos geoespaciales.
- Caché y almacenamiento temporal.
- Archivos y Object Storage.
- Embeddings.
- Búsqueda vectorial.
- RAG y LLM.
- Text-to-SQL.
- Persistencia políglota aplicada a IoT.
- Práctica integradora.



Vamos a recorrer desde la telemetría generada por un sensor hasta arquitecturas capaces de combinar datos estructurados, documentos e inteligencia artificial.

1

Datos IoT y Almacenamiento Especializado



Datos diferentes

Una misma solución puede tener necesidades muy distintas



Telemetría

Crece continuamente y está ordenada en el tiempo.



Estado actual

Necesita recuperarse rápidamente.



Ubicación

Requiere relaciones espaciales.



Documentos y archivos

Pueden tener varios megabytes o más.



Contenido textual

Puede necesitar búsqueda semántica.



La forma del dato y la forma de utilizarlo condicionan dónde y cómo conviene almacenarlo.

Antes de elegir una tecnología

Preguntar cómo vamos a utilizar los datos

Para cada conjunto de información podemos preguntarnos:

- ¿qué estructura tiene?;
- ¿con qué frecuencia se genera?;
- ¿cuánto puede crecer?;
- ¿cómo se consulta?;
- ¿cuánto tiempo debe conservarse?;
- ¿necesitamos conocer solamente el estado actual?;
- ¿existen relaciones temporales o espaciales?;
- ¿se trata de contenido binario?;
- ¿necesitamos buscar por significado?



La tecnología aparece después de comprender los datos y sus patrones de acceso.

¿Necesitamos otra tecnología?

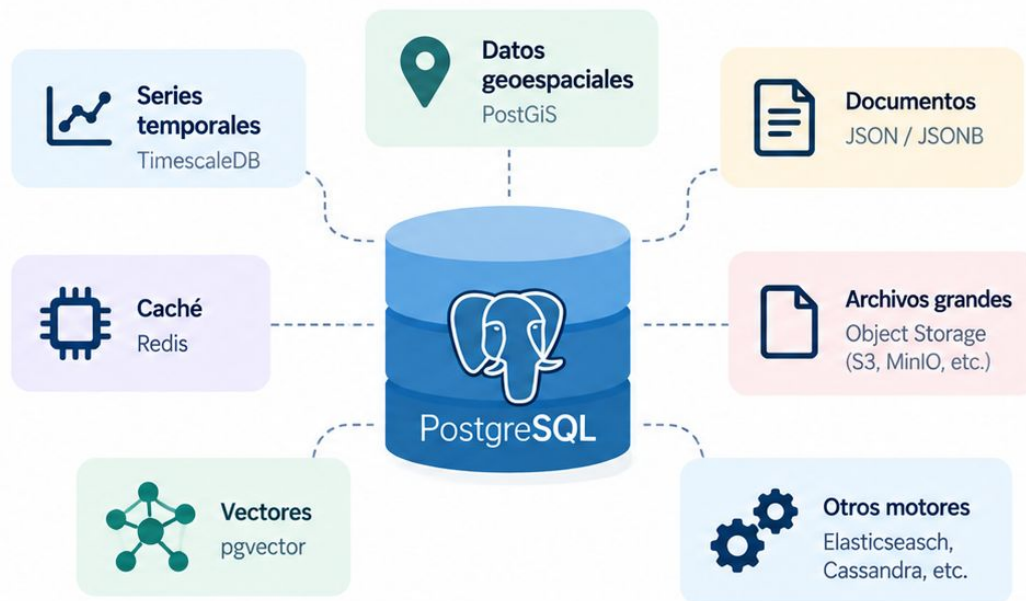
No necesariamente

PostgreSQL puede almacenar:

- dispositivos;
- mediciones;
- coordenadas;
- JSON;
- archivos pequeños;
- vectores.

Pero determinados requerimientos pueden justificar:

- extensiones especializadas;
- mecanismos adicionales;
- motores optimizados para una operación concreta.



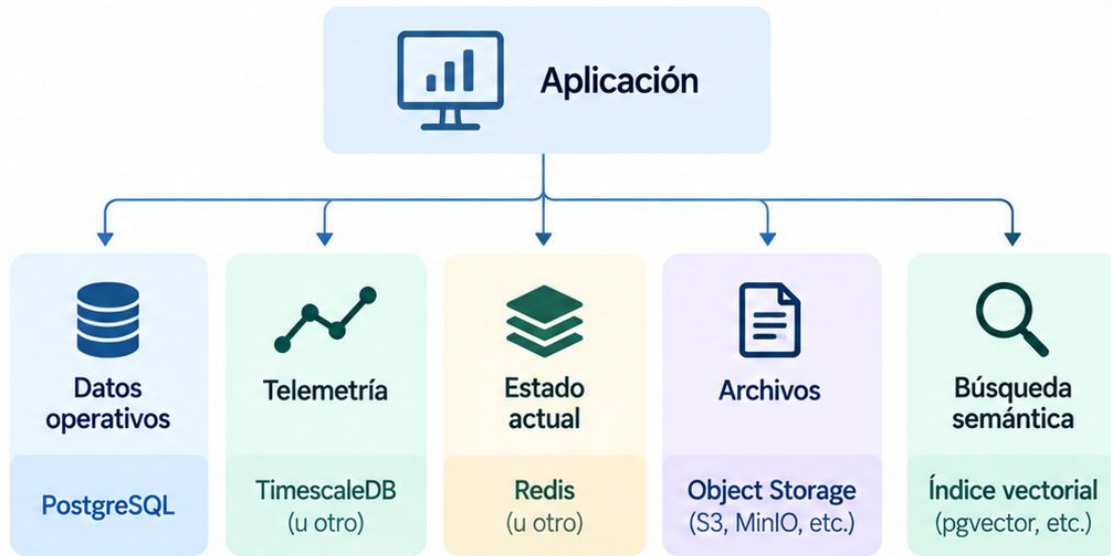
Que exista una tecnología especializada no significa que debamos incorporarla.

Especialización en la arquitectura

Cada almacenamiento puede resolver una necesidad concreta

Una misma aplicación puede utilizar distintos mecanismos **de** almacenamiento según el tipo de información:

- Datos operativos → PostgreSQL
- Telemetría → almacenamiento temporal especializado
- Estado actual → caché
- Archivos → Object Storage
- Búsqueda semántica → índice vectorial

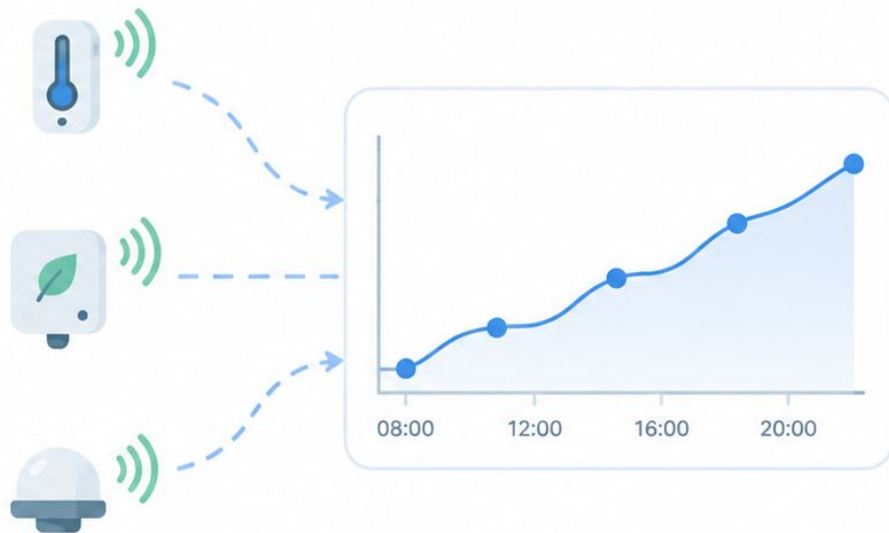


Persistencia polígota significa combinar mecanismos con una justificación, no acumular tecnologías.

2

Series Temporales

El tiempo forma parte central de la información.



Serie temporal

Observaciones asociadas a momentos del tiempo

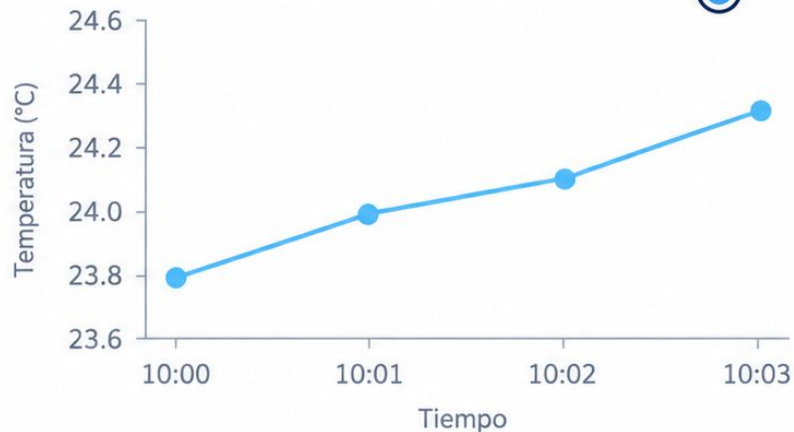
Ejemplo

10:00	→	23.8 °C
10:01	→	24.0 °C
10:02	→	24.1 °C
10:03	→	24.3 °C

Cada registro representa una observación realizada en un momento determinado.



Evolución de la temperatura



En una serie temporal, el tiempo no es solamente otro atributo: organiza la evolución de los datos.

Telemetría

Datos generados continuamente por dispositivos

Un dispositivo puede informar periódicamente:

- temperatura;
- humedad;
- presión;
- consumo;
- batería;
- nivel;
- velocidad;
- corriente;
- otros indicadores.

Cada nueva observación aumenta el histórico.



El dispositivo puede permanecer durante años. Las mediciones continúan creciendo.

Timestamp

¿Cuándo ocurrió la medición?

Una observación puede representarse como:

```
device_id = AMB-001  
variable  = temperatura  
valor     = 24.6  
timestamp = 2026-09-24 10:32:15
```

El timestamp permite:

- ordenar;
- consultar períodos;
- reconstruir evolución;
- calcular agregaciones temporales.



Sin tiempo podemos conocer un valor, pero no reconstruir su evolución.

Adquisición y recepción

Dos momentos que pueden ser diferentes



timestamp de adquisición

Cuándo ocurrió la medición.

Ejemplo: 2026-09-24 10:32:15



timestamp de recepción

Cuándo llegó al servidor.

Ejemplo: 2026-09-24 10:35:42



En sistemas IoT, el momento de generación y el momento de recepción pueden no coincidir.

Una serie necesita contexto

El valor y el tiempo no siempre alcanzan

Podemos necesitar conocer:

- dispositivo;
- sensor;
- variable;
- unidad;
- ubicación;
- modelo;
- organización.

AMB-001 + temperatura

AMB-001 + humedad

AMB-002 + temperatura

Cada combinación representa una evolución diferente.



AMB-001



AMB-001 + temperatura



AMB-001 + humedad



AMB-002



AMB-002 + temperatura



Una serie temporal combina dimensión temporal con información que identifica qué estamos observando.

Contexto + medición

Separar cómo identificamos de qué observamos



Las dimensiones permiten separar y agrupar series; el valor representa la observación.

¿Cómo modelamos la telemetría?

No existe una única estructura

Alternativa A:

timestamp	device	temperatura	humedad	presión
2024-03-01 10:00:00	dev-001	22.5	45	1013.2
2024-03-01 10:01:00	dev-001	22.7	44	1013.0
2024-03-01 10:02:00	dev-002	19.8	52	1009.8



Alternativa B:

timestamp	device	variable	valor
2024-03-01 10:00:00	dev-001	temperatura	22.5
2024-03-01 10:00:00	dev-001	humedad	45
2024-03-01 10:00:00	dev-001	presión	1013.2
2024-03-01 10:01:00	dev-001	temperatura	22.7

La elección depende de:

- variables disponibles;
- frecuencia de cambio;
- consultas;
- volumen;
- facilidad de evolución.



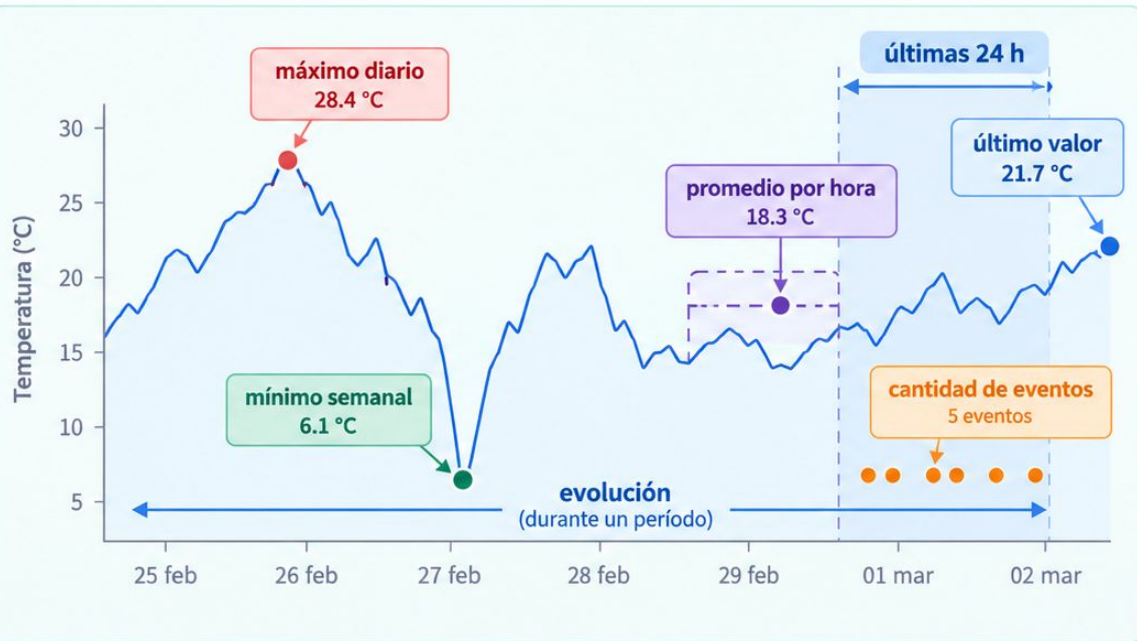
El modelo de telemetría también debe responder a los requerimientos del sistema.

Preguntas temporales

Cómo utilizamos el histórico

Podemos necesitar:

- último valor;
- mediciones de las últimas 24 horas;
- máximo diario;
- mínimo semanal;
- promedio por hora;
- cantidad de eventos;
- evolución durante un período.



Los patrones temporales de consulta son una característica central del almacenamiento de telemetría.

Ventanas de tiempo

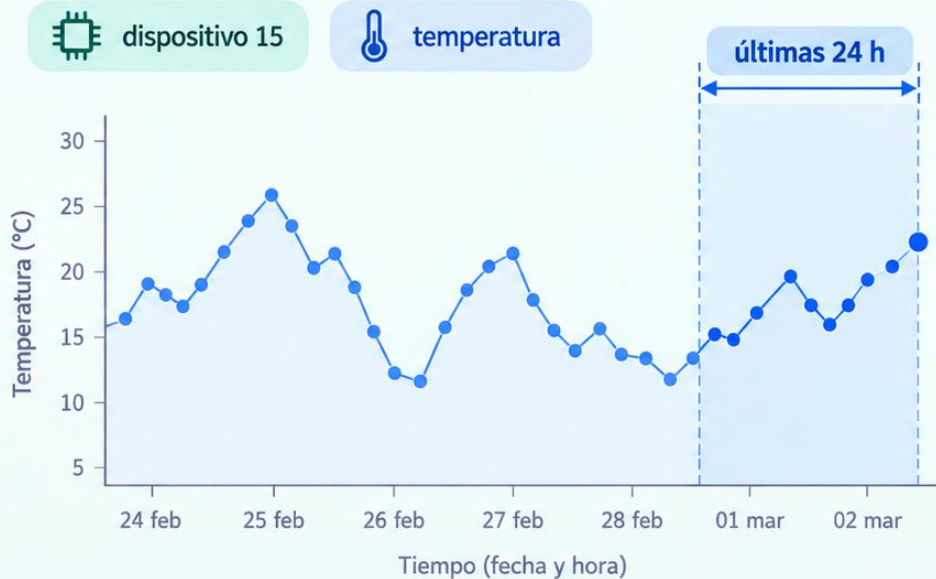
Una operación típica en IoT

Consulta SQL:

```
SELECT timestamp, valor
FROM mediciones
WHERE dispositivo_id = 15
    AND variable = 'temperatura'
    AND timestamp >= NOW() - INTERVAL '24 hours'
ORDER BY timestamp;
```

Buscamos:

un dispositivo + una variable + un intervalo temporal



Muchas consultas de telemetría recorren períodos, no registros aislados.

Series temporales en PostgreSQL

Especializar una estructura que ya conocemos

Podemos comenzar con una tabla PostgreSQL convencional:

```
CREATE TABLE mediciones (  
  dispositivo_id BIGINT NOT NULL,  
  timestamp TIMESTAMPTZ NOT NULL,  
  variable TEXT NOT NULL,  
  valor NUMERIC NOT NULL  
);
```

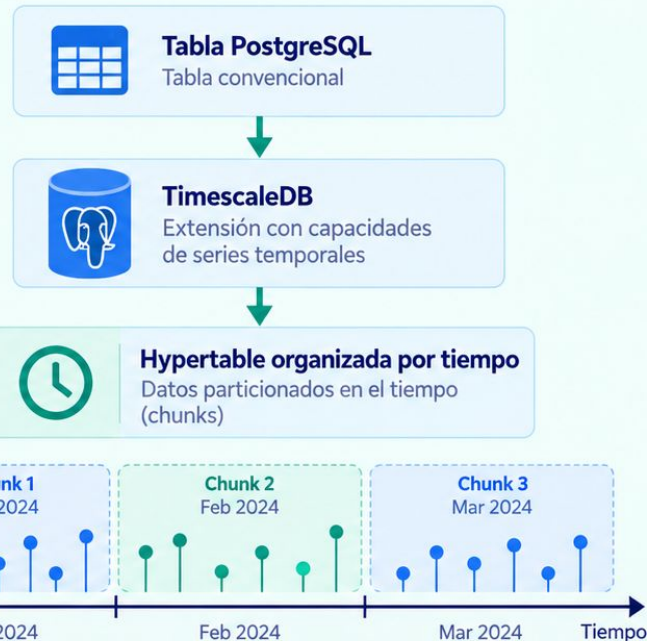
Una extensión como **TimescaleDB** incorpora capacidades orientadas a series temporales:

```
CREATE EXTENSION IF NOT EXISTS timescaledb;
```

Podemos organizar la tabla temporalmente:

```
SELECT create_hypertable(  
  'mediciones',  
  'timestamp'  
);
```

La aplicación puede continuar trabajando mediante SQL.

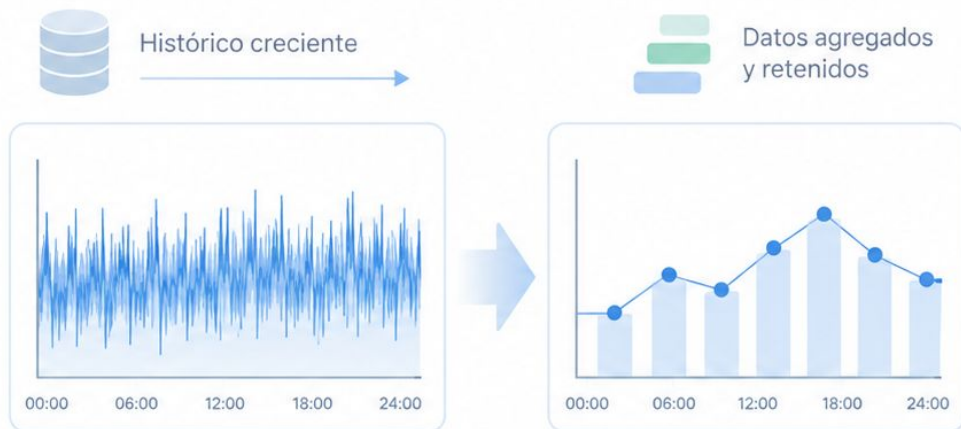


Una especialización temporal puede modificar cómo se organizan y administran los datos sin cambiar la forma general en que los consultamos.

3

Agregación, Retención y Downsampling

Gestionar un histórico que crece permanentemente.



El volumen crece rápido

Una pequeña frecuencia se vuelve un gran volumen

Multiplicar antes de diseñar

Un dispositivo enviando:

1 medición / minuto

produce aproximadamente:

525.600 mediciones / año

Con 1.000 dispositivos:

525.600.000 mediciones / año

Y eso considerando solamente una variable.



La frecuencia de adquisición determina directamente la velocidad de crecimiento del histórico.

Resumir información por períodos

Promedio, mínimo y máximo a lo largo del tiempo

Podemos agrupar mediciones por hora utilizando PostgreSQL:

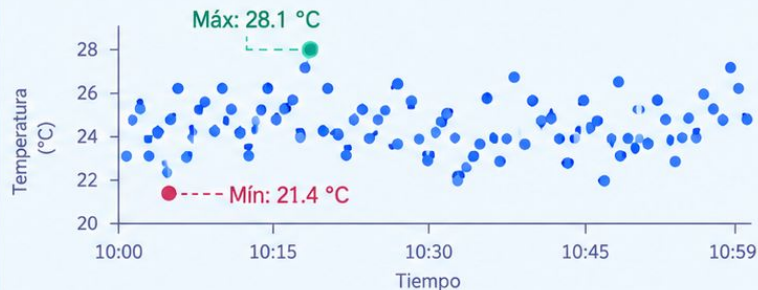
```
SELECT
    date_trunc('hour', timestamp) AS hora,
    AVG(valor) AS promedio,
    MIN(valor) AS minimo,
    MAX(valor) AS maximo
FROM mediciones
WHERE dispositivo_id = 15
    AND variable = 'temperatura'
GROUP BY hora
ORDER BY hora;
```

Con una extensión temporal podemos expresar el mismo concepto mediante ventanas:

```
SELECT
    time_bucket('1 hour', timestamp) AS hora,
    AVG(valor) AS promedio
FROM mediciones
GROUP BY hora
ORDER BY hora;
```

Muchas mediciones dentro de una hora

Mediciones de 10:00 a 10:59 (60 datos, aprox.)



Resultado resumido por hora

10:00 → promedio 24.7 °C

(mínimo 21.4 °C, máximo 28.1 °C)



Una agregación temporal transforma muchas observaciones en información resumida para cada período.

Reducir resolución

Conservar menos puntos sin perder la tendencia general

Supongamos que almacenamos:

1 medición / minuto

```
SELECT
    time_bucket('5 minutes', timestamp) AS intervalo,
    AVG(valor) AS valor_promedio
FROM mediciones
WHERE dispositivo_id = 15
    AND variable = 'temperatura'
GROUP BY intervalo
ORDER BY intervalo;
```

Conceptualmente:

60 mediciones / hora

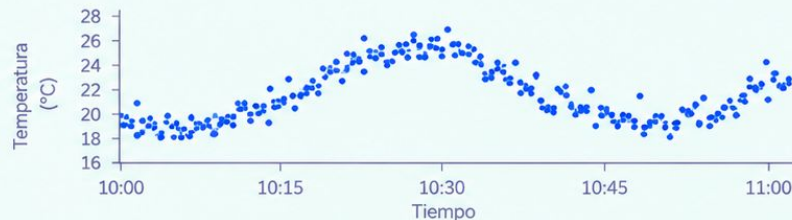


12 valores / hora

Los datos agregados ocupan menos espacio y permiten trabajar con históricos extensos a una resolución menor.

Datos originales

1 medición / minuto (alta resolución temporal)



Datos reducidos

1 valor cada 5 minutos (resolución reducida)



60 mediciones / hora → 12 valores / hora



Downsampling reduce la resolución temporal conservando una representación del comportamiento de los datos.

Políticas de retención

Definir cuánto tiempo conservamos cada nivel de detalle

Ejemplo de política:



Datos crudos

Mayor detalle, más reciente

30 días



Agregado cada 5 minutos

Detalle medio

6 meses



Agregado horario

Menor detalle, largo plazo

5 años

Con TimescaleDB podemos definir una política de retención:

```
SELECT add_retention_policy(  
  'mediciones',  
  INTERVAL '30 days'  
);
```

Las mediciones que excedan ese período podrán eliminarse automáticamente según la política configurada.

Evolución de la resolución temporal

A mayor antigüedad, menor nivel de detalle



Retener datos también significa decidir cuándo determinada información deja de justificar su costo de almacenamiento.

Dos problemas diferentes

Ciclo de vida $\neq$ recuperación ante fallos



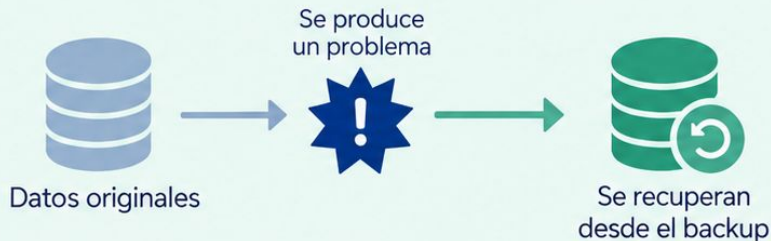
Retención

Define durante cuánto tiempo queremos conservar información operativa.



Backup

Permite recuperar datos frente a pérdida, corrupción o errores.



Eliminar datos porque terminó su período de retención no significa que ocurrió un fallo.



Retención decide qué datos deben existir. Backup permite recuperarlos cuando algo sale mal.

4

Datos Geoespaciales

Representar dónde se encuentra algo y qué relación espacial existe entre distintos elementos.



Representar una ubicación

De dos números a información espacial

Primero habilitamos las capacidades espaciales:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

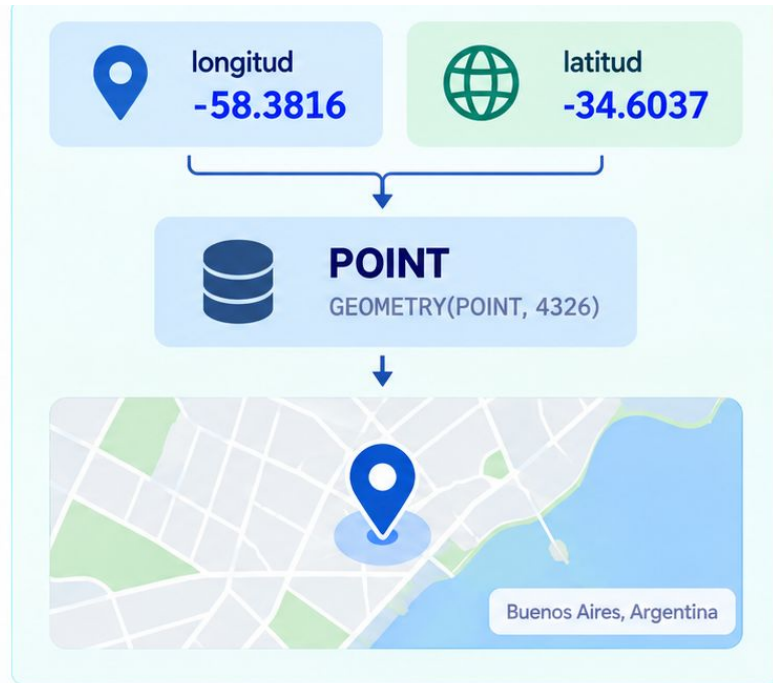
Podemos incorporar una geometría al dispositivo:

```
ALTER TABLE dispositivos  
ADD COLUMN ubicacion  
GEOMETRY(POINT, 4326);
```

Y almacenar un punto:

```
UPDATE dispositivos  
SET ubicacion =  
    ST_SetSRID(  
        ST_MakePoint(-58.3816, -34.6037),  
        4326  
    )  
WHERE dispositivo_id = 15;
```

`ST_MakePoint` recibe primero longitud y luego latitud.



Una coordenada puede transformarse en un tipo espacial que el motor puede interpretar y consultar.

Geometrías

Diferentes objetos espaciales



Punto

Un dispositivo.



Línea

Un recorrido.



Polígono

Una zona de cobertura o región.

Podemos necesitar representar no solamente posiciones sino formas y áreas.



Los datos geospaciales incorporan geometría además de atributos tradicionales.

Dispositivos cercanos

Consultar utilizando distancia

Necesitamos encontrar dispositivos ubicados a menos de 2 km de un punto determinado.

Podemos consultar:

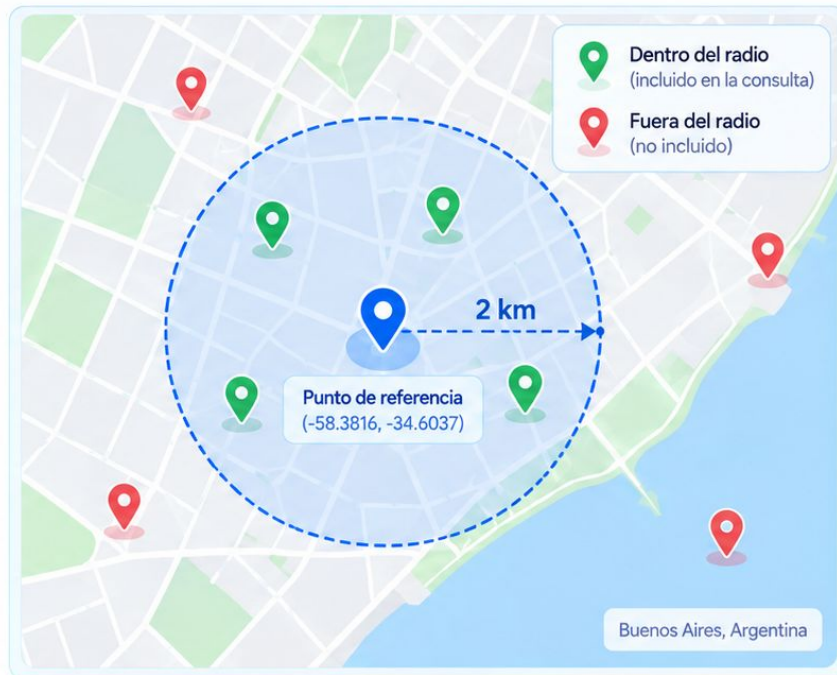
```
SELECT
    dispositivo_id,
    nombre
FROM dispositivos
WHERE ST_DWithin(
    ubicacion::geography,
    ST_SetSRID(
        ST_MakePoint(-58.3816, -34.6037),
        4326
    )::geography,
    2000
);
```

En este ejemplo:

2000 → metros

La condición ya no compara solamente valores.

Evalúa una relación espacial.



Con capacidades geoespaciales podemos expresar distancia y proximidad directamente dentro de una consulta.

¿Está dentro de esta zona?

Relacionar puntos con polígonos

Supongamos una tabla de zonas:

zonas
zona_id
nombre
geometria

Podemos consultar qué dispositivos se encuentran dentro de una determinada región:

```
SELECT
  d.dispositivo_id,
  d.nombre
FROM dispositivos d
JOIN zonas z
  ON ST_Contains(
    z.geometria,
    d.ubicacion
  )
WHERE z.nombre = 'Zona Norte';
```



Las relaciones espaciales permiten consultar ubicación, pertenencia e intersección sin reducir el problema a comparaciones de coordenadas individuales.

Capacidades geoespaciales

Una extensión incorpora nuevos tipos y operaciones

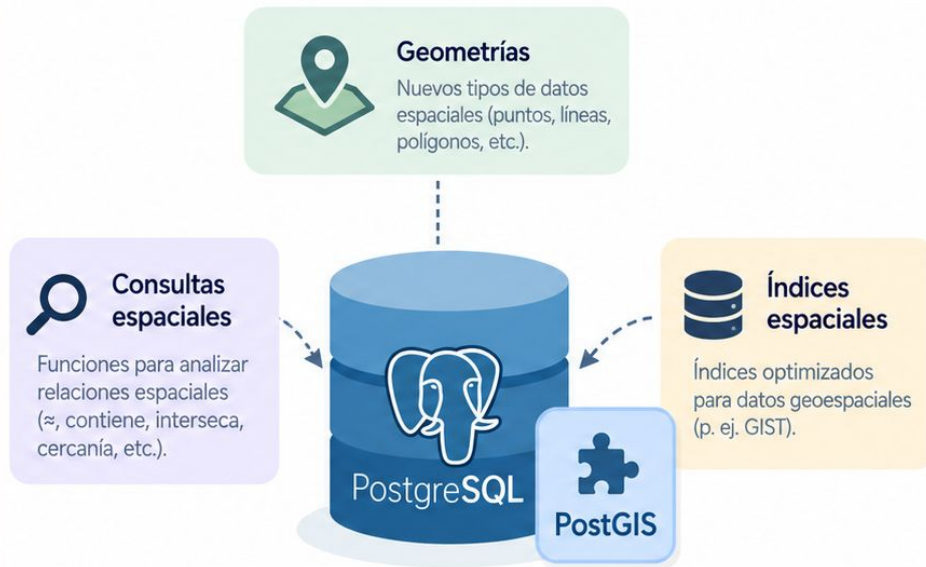
PostGIS agrega a PostgreSQL funciones como:

```
ST_Distance(...)  
ST_DWithin(...)  
ST_Contains(...)  
ST_Intersects(...)
```

También podemos crear índices espaciales:

```
CREATE INDEX idx_dispositivos_ubicacion  
ON dispositivos  
USING GIST (ubicacion);
```

Estos índices están diseñados para ayudar en consultas espaciales.

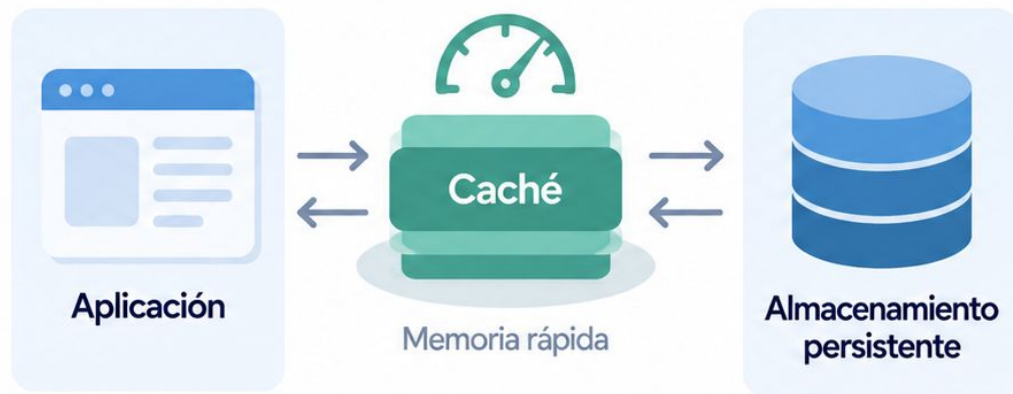


Una extensión especializada puede incorporar nuevos tipos, funciones e índices manteniendo el resto del modelo dentro de PostgreSQL.

5

Caché y Almacenamiento Temporal

No todos los datos necesitan la misma duración ni el mismo acceso.



Estado actual e histórico

Histórico en la base y estado actual en memoria

El histórico puede permanecer en PostgreSQL:

```
SELECT *  
FROM mediciones  
WHERE dispositivo_id = 15  
ORDER BY timestamp DESC;
```

Mientras el último estado puede mantenerse mediante una clave:

```
SET device:AMB-001:last_state ONLINE
```

Y recuperarse directamente:

```
GET device:AMB-001:last_state
```

Resultado:

```
"ONLINE"
```



Podemos conservar el histórico de manera persistente y mantener una representación optimizada del estado actual.

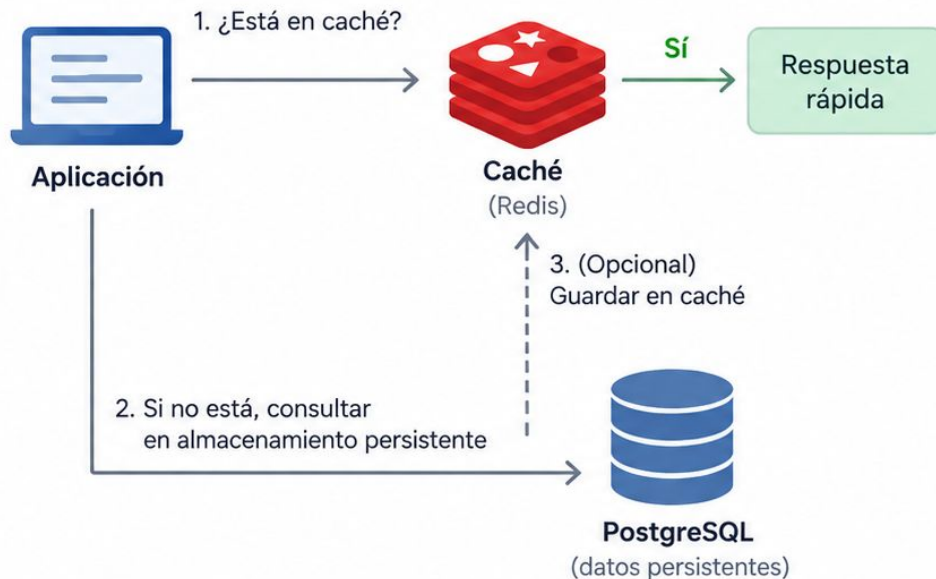
Caché

Mantener información de acceso frecuente

Una caché puede guardar temporalmente:

- último estado;
- última medición;
- configuraciones consultadas frecuentemente;
- resultados calculados;
- sesiones;
- datos externos recientemente obtenidos.

Su objetivo principal suele ser reducir el costo o la latencia de accesos repetidos.



La caché evita repetir determinadas operaciones cuando podemos reutilizar temporalmente un resultado.

Datos que pueden expirar

Time To Live

Podemos guardar un estado indicando cuánto tiempo debe existir:

```
SET device:AMB-001:last_state ONLINE EX 300
```

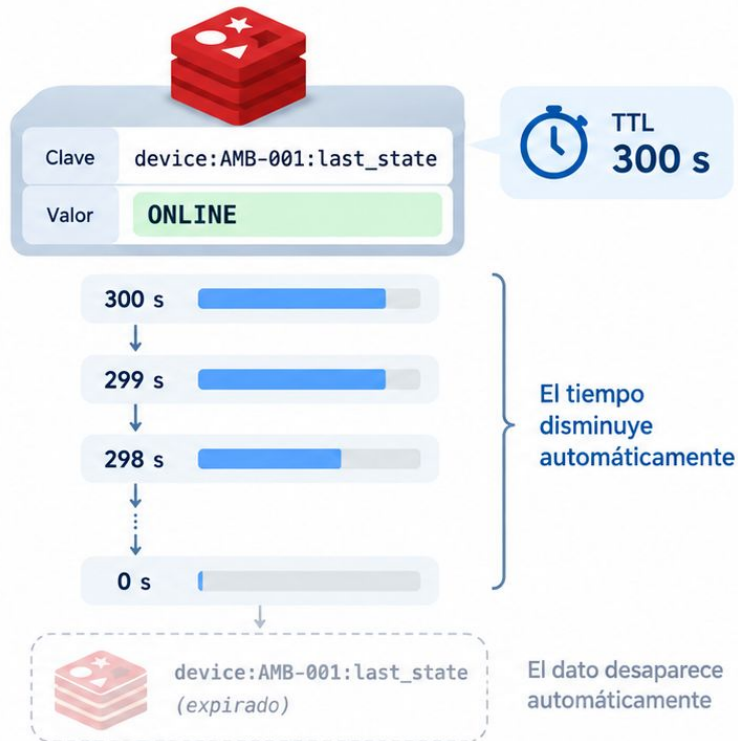
`EX 300` indica una expiración de 300 segundos.

Podemos consultar cuánto tiempo queda:

```
TTL device:AMB-001:last_state
```

Conceptualmente:

```
300
↓
299
↓
298
↓
...
↓
expira
```



TTL permite incorporar el tiempo de vida como parte del comportamiento del dato.

Estado temporal estructurado

Una clave puede contener varios campos

Podemos mantener información actual del dispositivo mediante un hash:

```
HSET device:AMB-001 \  
  status ONLINE \  
  temperature 24.6 \  
  humidity 58
```

Luego consultar:

```
HGETALL device:AMB-001
```

Resultado conceptual:

```
status      ONLINE  
temperature 24.6  
humidity    58
```

El histórico continúa almacenado en su mecanismo correspondiente.



Redis
(estado temporal)

device:AMB-001

status	ONLINE
temperature	24.6
humidity	58



PostgreSQL
(histórico persistente)

mediciones

timestamp	dispositivo_id	temperatura	humedad	...
2026-09-10 10:00	1	24.1	56	...
2026-09-10 10:05	1	24.3	57	...
2026-09-10 10:10	1	24.6	58	...
...	...	...	...	...

—•—•—•—•—•—→
Datos históricos en el tiempo



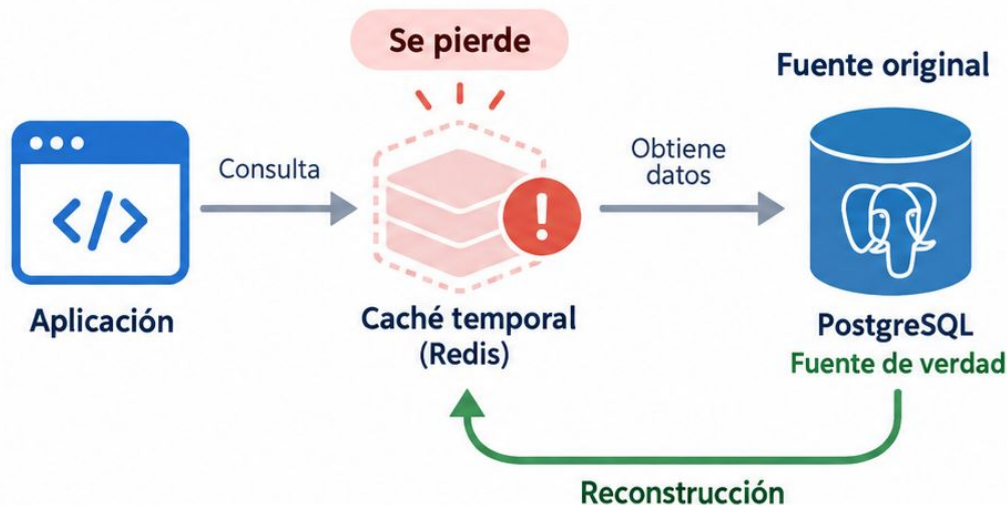
Una representación temporal puede organizar varios atributos actuales sin reemplazar el histórico persistente.

¿Qué ocurre si desaparece?

Pensar qué información podemos reconstruir

Cuando utilizamos una caché debemos preguntarnos:

- ¿qué ocurre si se pierde?
- ¿podemos reconstruir el dato?
- ¿cuál es la fuente original?
- ¿cuándo debe invalidarse?
- ¿qué duración es razonable?

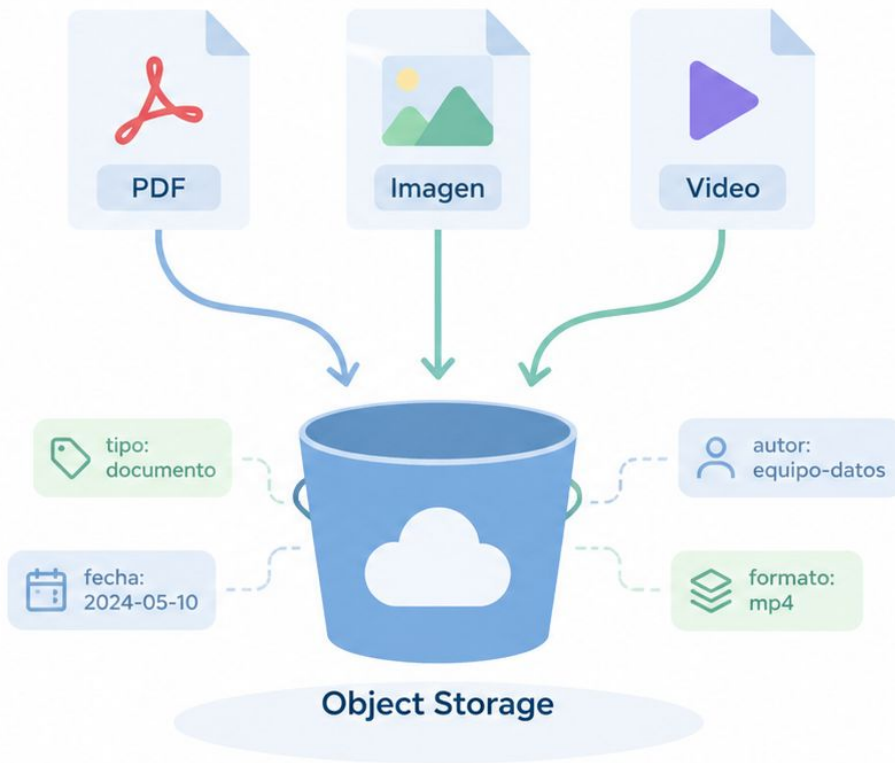


Antes de almacenar algo temporalmente debemos saber de dónde volveríamos a obtenerlo.

6

Archivos y Object Storage

Separar contenido físico de la información que lo describe.



Archivos dentro de una solución

Algunos datos tienen otra naturaleza

Una plataforma puede administrar:

- fotografías;
- audio;
- video;
- manuales PDF;
- firmware;
- reportes;
- archivos de configuración;
- logs extensos.

Registros tradicionales			
id	dispositivo	temperatura	fecha
1	Sensor-01	22.4	2024-04-01
2	Sensor-02	19.8	2024-04-01
3	Sensor-01	21.1	2024-04-02
4	Sensor-03	23.5	2024-04-02
...	...	...	...



Objetos binarios (más grandes)



Manuales
PDF



Fotografías



Audio



Video



Firmware



Logs
extensos

Estos archivos pueden ser mucho más grandes que los registros tradicionales.



Una arquitectura de datos también debe decidir dónde viven los objetos binarios.

Contenido y metadatos

Dos responsabilidades diferentes

Podemos almacenar archivos dentro de algunos SGBD.
Pero también podemos separar:

Contenido físico



manual.pdf



foto.jpg



firmware.bin

Información sobre el contenido



nombre



tipo



tamaño



propietario



fecha



versión



ubicación



Archivos



Repositorio de objetos
(contenido físico)

Metadatos

nombre	manual.pdf
tipo	PDF
tamaño	2,4 MB
propietario	ana
fecha	2024-03-12
versión	1.0
ubicación	/documentos/manual.pdf



Separar contenido y metadatos permite administrar cada parte mediante mecanismos adecuados.

Almacenamiento de objetos

Guardar archivos mediante una clave

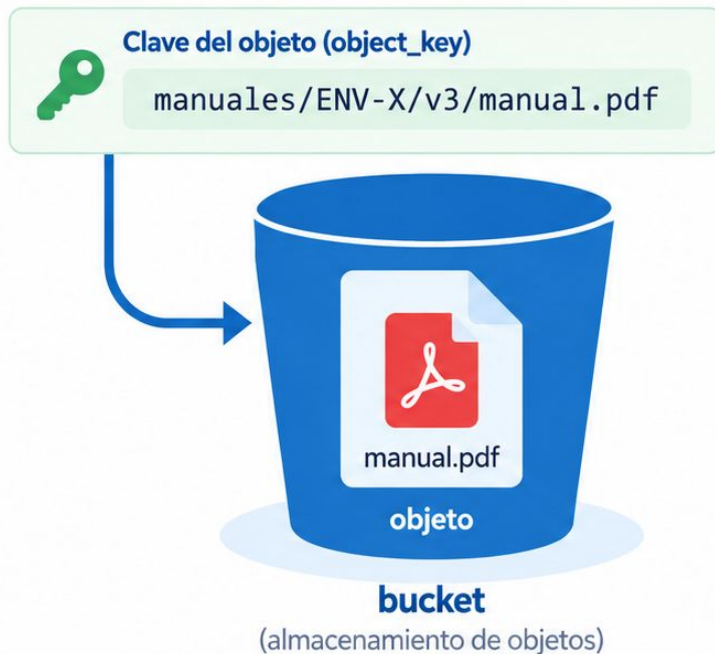
Conceptualmente:

object_key
↓
objeto

Ejemplo:

manuales/ENV-X/v3/manual.pdf

El almacenamiento de objetos está pensado para gestionar gran cantidad de contenido identificado mediante claves.



El archivo puede almacenarse como objeto mientras la aplicación conserva sus relaciones y metadatos en otra estructura.

Almacenar el objeto

SeaweedFS como ejemplo compatible con S3

Supongamos el archivo:

manual_ENV_X_v3.pdf

Podemos almacenarlo dentro de un bucket:

documentos

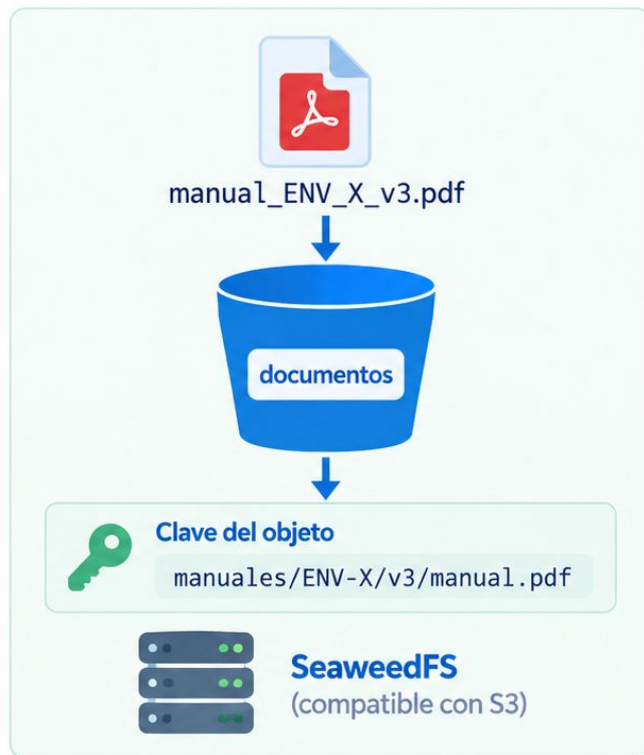
con una clave:

manuales/ENV-X/v3/manual.pdf

Usando el siguiente comando:

```
aws s3 cp manual_ENV_X_v3.pdf \  
s3://documentos/manuales/ENV-X/v3/manual.pdf \  
--endpoint-url http://seaweedfs:8333
```

El contenido físico queda almacenado en SeaweedFS.



Object Storage identifica el contenido mediante una clave; la aplicación puede conservar esa referencia junto con sus metadatos.

Relacionar el archivo

Relacionar objetos con el resto del sistema

Podemos crear una estructura para sus metadatos:

```
CREATE TABLE documentos (  
  documento_id BIGINT  
    GENERATED ALWAYS AS IDENTITY  
    PRIMARY KEY,  
  nombre TEXT NOT NULL,  
  tipo TEXT NOT NULL,  
  version TEXT,  
  object_key TEXT NOT NULL UNIQUE,  
  creado_en TIMESTAMPTZ  
    DEFAULT NOW()  
);
```

Registramos el objeto:

```
INSERT INTO documentos  
(nombre, tipo, version, object_key)  
VALUES (  
  'Manual ENV-X',  
  'manual',  
  '3',  
  'manuales/ENV-X/v3/manual.pdf'  
);
```



PostgreSQL

documentos

documento_id	1
nombre	Manual ENV-X
tipo	manual
version	3
object_key	manuales/ENV-X/v3/manual.pdf



SeaweedFS

(almacenamiento de objetos)



manual.pdf

manuales/ENV-X/v3/manual.pdf



SeaweedFS almacena el contenido. PostgreSQL puede representar qué es, cómo se relaciona y dónde encontrarlo.

Fuente original

Conservar el objeto del cual proviene la información

Tenemos:



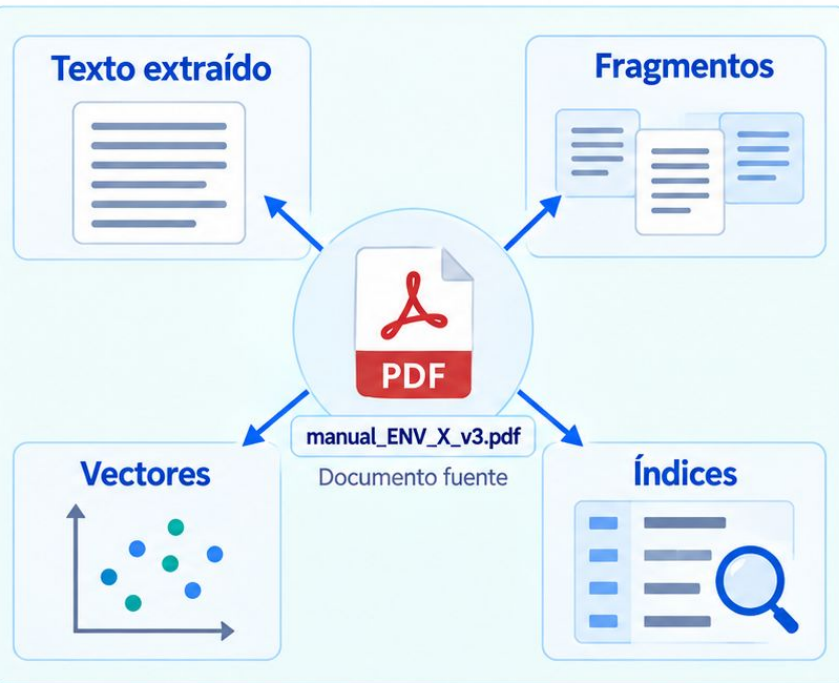
manual_ENV_X_v3.pdf

Ese documento constituye el **contenido original**.

De él podemos crear:

- texto extraído;
- fragmentos;
- vectores;
- índices.

Pero todos ellos derivan del **documento fuente**.



Las representaciones derivadas no reemplazan necesariamente al documento original.

Procesar un documento

Transformar contenido para poder buscarlo

Un PDF almacenado en Object Storage puede necesitar procesamiento antes de poder utilizar su contenido.

El resultado es una representación derivada.



Almacenar un documento y poder consultar su contenido son problemas distintos.

Fragmentar el documento

Mantener cada fragmento (chunk) relacionado con su documento

Podemos representar los fragmentos mediante una tabla:

```
CREATE TABLE document_chunks (  
  chunk_id BIGINT  
    GENERATED ALWAYS AS IDENTITY  
    PRIMARY KEY,  
  
  documento_id BIGINT NOT NULL,  
  chunk_index INTEGER NOT NULL,  
  pagina INTEGER,  
  seccion TEXT,  
  texto TEXT NOT NULL,  
  
  FOREIGN KEY (documento_id)  
    REFERENCES documentos(documento_id)  
);
```

Relación:

documentos
1
|
N
document_chunks

Un documento puede generar múltiples fragmentos.

documentos	
documento_id	nombre
1	Manual_RAG.pdf
...	...

1 : N

document_chunks					
chunk_id	documento_id	chunk_index	pagina	seccion	texto
1	1	0	1	"Introducción"	RAG permite...
2	1	1	1	"Arquitectura"	"La arquitectura...
3	1	2	2	"Indexación"	Para indexar...
4	1	3	2	"Consultas"	"Las consultas...
...	...	...	...	...	...



Chunk 0 (p. 1)

RAG permite combinar la búsqueda...



Chunk 1 (p. 1)

La arquitectura de un sistema RAG...



Chunk 2 (p. 2)

Para indexar los documentos es...



Chunk 3 (p. 2)

Las consultas se procesan mediante...



RAG también necesita modelado: cada chunk debe conservar su relación con el documento del cual fue generado.

Un nuevo tipo de búsqueda

Ahora queremos recuperar contenido por significado

Supongamos que el manual contiene:



“Restablecimiento de comunicaciones
luego de una interrupción de red.”

Y el usuario pregunta:



“¿Cómo recupero la conectividad
del dispositivo?”

Las palabras no son exactamente las mismas.

Pero las ideas están relacionadas.

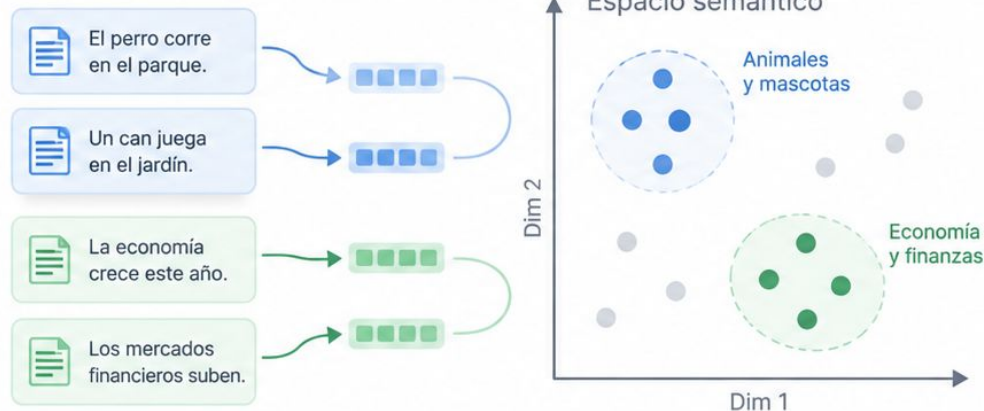


Para buscar por significado necesitamos otra representación del contenido.

7

Embeddings y Búsqueda Vectorial

Representar contenido para poder comparar su similitud.



Búsqueda tradicional

Encontrar coincidencias textuales

Consulta:

"error de batería"

Podemos buscar documentos donde aparezcan:

- error
- batería

Esto funciona muy bien cuando conocemos los términos que buscamos.

Pero no todas las preguntas utilizan las mismas palabras que el documento.

Q error de batería



Coincidencia textual y similitud semántica no representan exactamente el mismo problema.

Generar un embedding

El modelo transforma texto en una representación numérica

Partimos de un fragmento:

```
Restablecer la conectividad Wi-Fi  
del dispositivo.
```

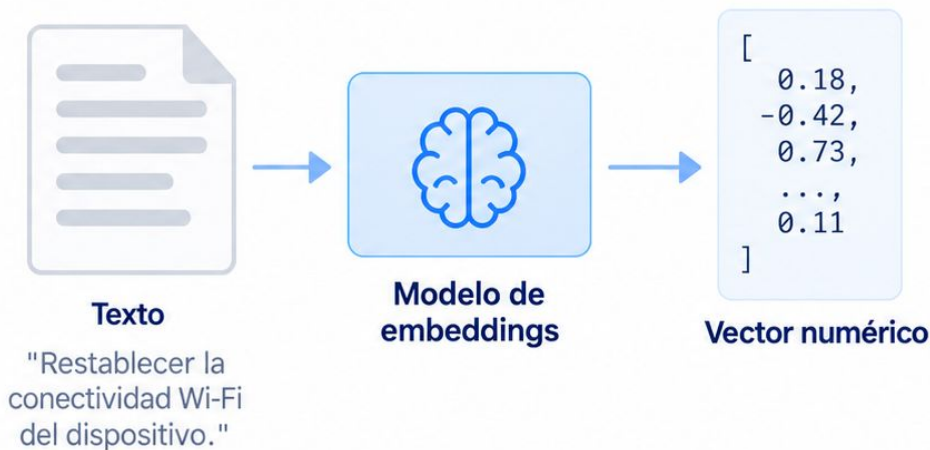
Conceptualmente:

```
texto = ""  
Restablecer la conectividad Wi-Fi  
del dispositivo.  
""  
  
embedding = embedding_model(texto)
```

Resultado:

```
[  
  0.18,  
 -0.42,  
  0.73,  
  ...,  
  0.11  
]
```

El número de dimensiones depende del modelo utilizado.



El modelo de embeddings no genera una respuesta: transforma contenido en una representación que podemos almacenar y comparar.

¿Qué hace el modelo?

Representar, no responder

Un modelo de embeddings recibe contenido y produce un vector.

Por ejemplo:

texto
↓
modelo
↓
vector

Ese vector puede utilizarse para comparar contenidos.

El modelo de embeddings no está generando una respuesta para el usuario.

Embedding Model

Representa contenido como un vector numérico



Texto
"Un artículo sobre bicicletas."



Modelo de embeddings



[
0.21,
-0.08,
0.55,
...,
-0.31
]

Vector numérico

LLM

Genera texto (etapa posterior)



Contenido recuperado
"Un artículo sobre bicicletas."



LLM



Texto generado
"Aquí tienes un resumen..."



El modelo de embeddings representa contenido.

Más adelante, un LLM podrá utilizar contenido recuperado para generar una respuesta.

Espacio vectorial

Contenido similar, vectores cercanos

Conceptualmente:

“restablecer Wi-Fi”



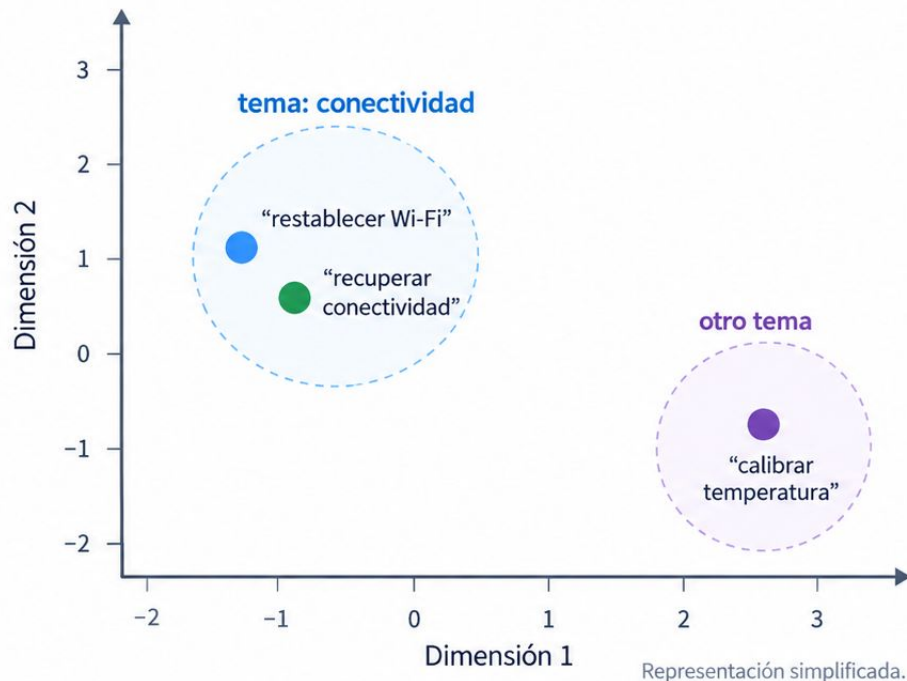
“recuperar conectividad”



“calibrar temperatura”



Las dos primeras expresiones tienen significados relacionados. La tercera representa otro tema.



La posición relativa entre vectores permite aproximar qué contenidos resultan similares.

Comparar vectores

Buscar representaciones cercanas

Pregunta:

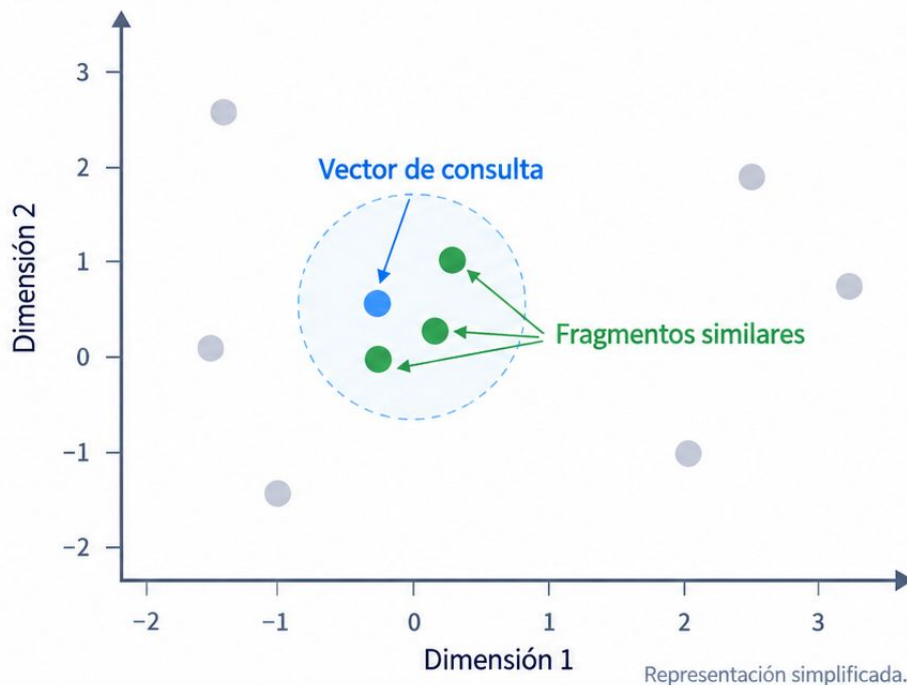
¿Cómo recupero la conexión?

Vector de consulta

Comparación con vectores almacenados

Fragmentos más similares

Podemos utilizar métricas como la similitud de coseno para comparar las representaciones.



La búsqueda vectorial recupera elementos según proximidad en el espacio de representación.

¿Por qué fragmentar?

Un vector para todo el documento puede ser demasiado general

Supongamos un manual de 80 páginas sobre:

- instalación;
- conectividad;
- calibración;
- mantenimiento;
- diagnóstico.

Representarlo como una única unidad dificulta recuperar una sección específica.

Dividirlo permite encontrar solamente las partes relacionadas con la pregunta.

Documento completo

Un solo vector (demasiado general)



80 páginas

Un único vector
(temas mezclados)

Documento dividido en fragmentos

Vectores más específicos



Varios vectores
(temas específicos)



La unidad que almacenamos no necesariamente debe ser la misma unidad que utilizamos para recuperar información.

¿Cuánto contenido incluimos?

Un equilibrio entre precisión y contexto

Chunk muy grande



Gran tamaño

Mucho contenido potencialmente irrelevante.

Chunk muy pequeño



Pequeño tamaño

Puede perder información necesaria para interpretar el fragmento.

Chunk adecuado



Tamaño equilibrado

Conserva suficiente contexto sin abarcar demasiado contenido.



El tamaño del fragmento también es una decisión de arquitectura de datos.

Mantener el origen

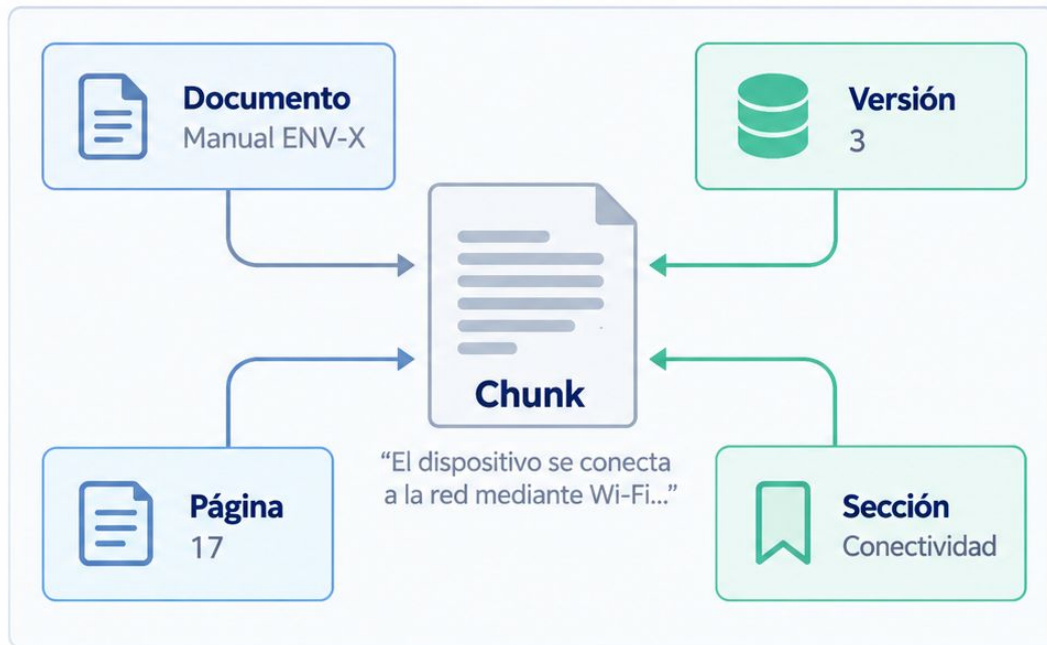
Un chunk necesita contexto, saber de dónde salió

Podemos conservar:

- chunk_id
- documento_id
- pagina
- seccion
- version
- texto

Ejemplo:

```
documento = Manual ENV-X  
version   = 3  
pagina    = 17  
seccion   = Conectividad
```



Recuperar un fragmento sin conocer su origen limita nuestra capacidad de interpretarlo y citarlo.

Guardar vectores

pgvector agrega un nuevo tipo de dato

Habilitamos la extensión:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Luego incorporamos el vector a nuestros chunks:

```
ALTER TABLE document_chunks  
ADD COLUMN embedding VECTOR(768);
```

Conceptualmente:

document_chunks
chunk_id
documento_id
texto
pagina
seccion
embedding

← 768 es solamente un ejemplo.

La dimensión debe coincidir con el modelo de embeddings utilizado.



El esquema debe conocer la dimensión del vector que produce el modelo seleccionado.

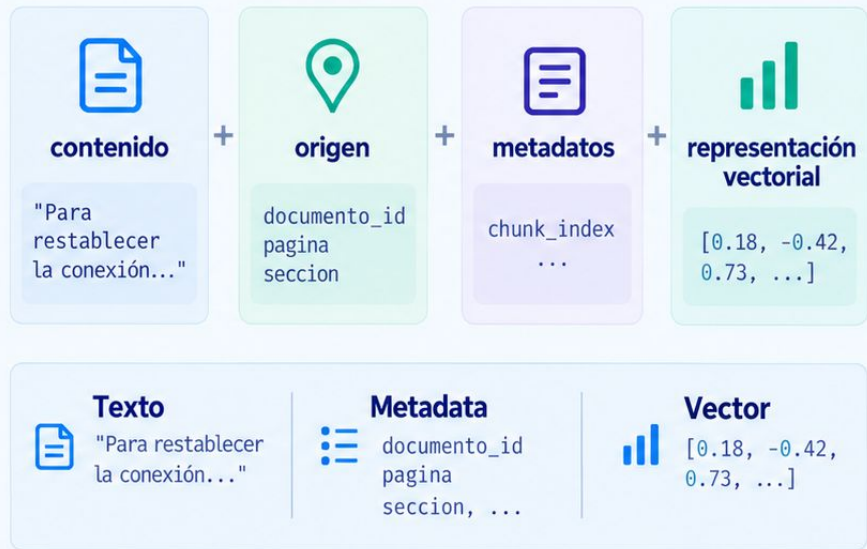
PostgreSQL + pgvector

El embedding acompaña al fragmento, no reemplaza su contenido

Podemos almacenar:

```
INSERT INTO document_chunks (  
  documento_id,  
  chunk_index,  
  pagina,  
  seccion,  
  texto,  
  embedding  
)  
VALUES (  
  12,  
  8,  
  17,  
  'Conectividad',  
  'Para restablecer la conexión...',  
  '[0.18, -0.42, 0.73, ...]'  
);
```

Ahora el mismo registro contiene:



El vector permite comparar. El texto sigue siendo la información que posteriormente queremos recuperar.

Metadatos + similitud

Encontrar los fragmentos más próximos a una pregunta

La pregunta también se transforma en un embedding:

```
[0.21, -0.39, 0.69, ...]
```

Podemos buscar:

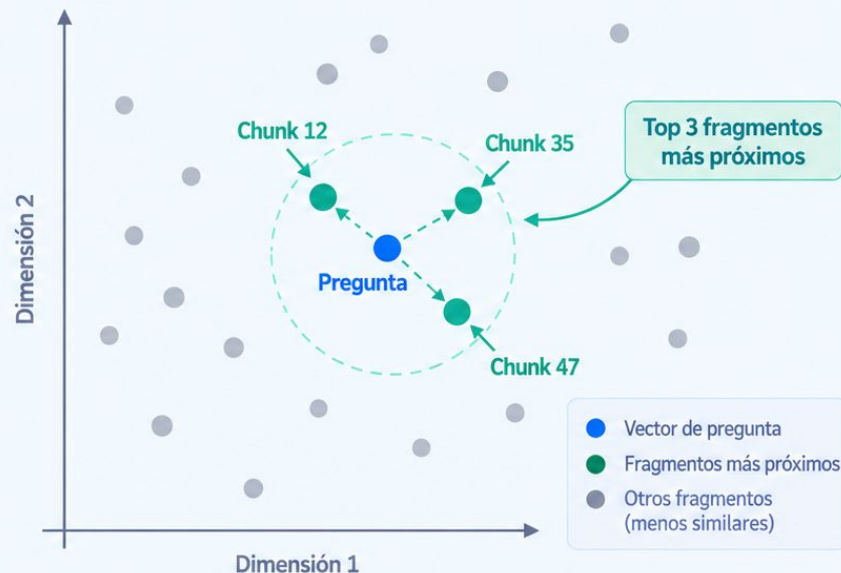
```
SELECT
  chunk_id,
  texto
FROM document_chunks
ORDER BY embedding <=>
  '[0.21, -0.39, 0.69, ...]'
LIMIT 3;
```

En pgvector:

<=> → distancia coseno

Los primeros resultados representan los vectores más cercanos según esa métrica.

Representación en un espacio vectorial (2D):



La búsqueda vectorial ordena los fragmentos según similitud con la representación de la consulta.

Búsqueda híbrida

Usar cada mecanismo para lo que resuelve mejor

Supongamos que conocemos:

```
modelo = ENV-X  
idioma = es  
version = 3
```

Podemos filtrar primero y luego ordenar por similitud:

```
SELECT  
  c.chunk_id,  
  c.texto  
FROM document_chunks c  
JOIN documentos d  
  ON d.documento_id = c.documento_id  
WHERE d.nombre = 'Manual ENV-X'  
      AND d.version = '3'  
ORDER BY c.embedding <=> :query_embedding  
LIMIT 5;
```

Tenemos dos operaciones diferentes:

SQL

→ filtra estructuralmente

Vector

→ ordena semánticamente

Gran conjunto de chunks

(por ejemplo, miles)



filtro SQL

subconjunto

(por ejemplo, decenas)



búsqueda vectorial

Top 5

(más similares)



Los vectores complementan los datos estructurados. No los reemplazan.

8

RAG y Modelos de Lenguaje

Recuperar información y utilizarla como contexto para generar una respuesta.

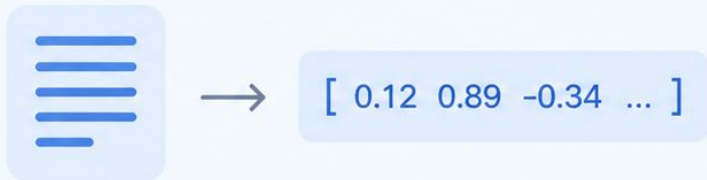


Recuperar y generar

No necesitamos el mismo modelo para todo

Modelo de embeddings

texto → vector



Permite representar y comparar contenido.

LLM

instrucciones + contexto → texto



Permite interpretar información y generar una respuesta.



El modelo de embeddings ayuda a encontrar información.
El LLM utiliza información para construir una respuesta.

¿Qué es un LLM?

Un modelo capaz de trabajar con lenguaje

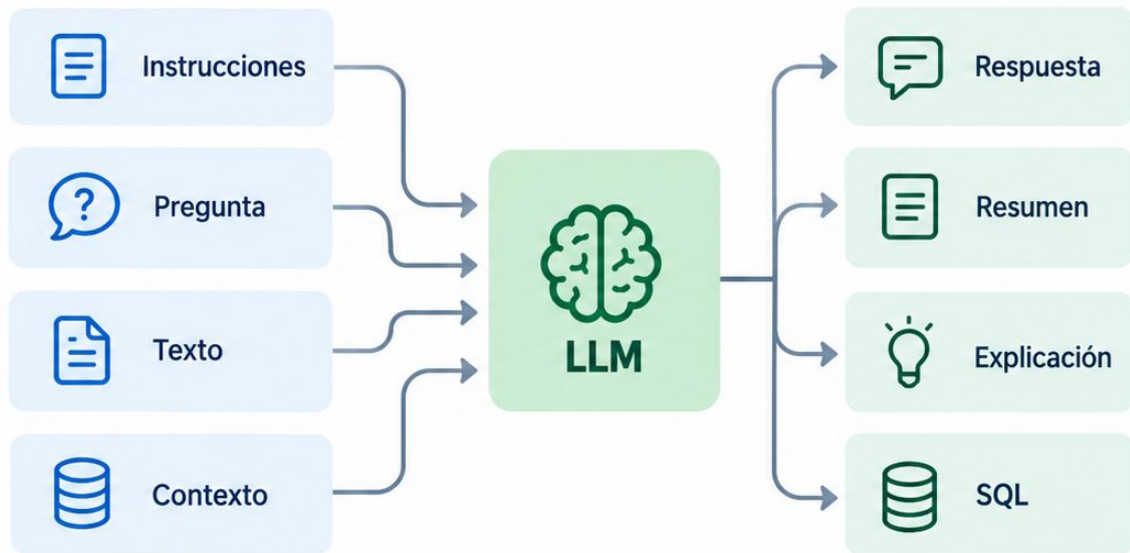
Un LLM puede recibir:

- instrucciones;
- preguntas;
- texto;
- contexto adicional.

Y generar:

- respuestas;
- resúmenes;
- explicaciones;
- estructuras;
- incluso consultas SQL.

Pero la respuesta depende de la información disponible en su contexto y de las capacidades del modelo.



Un LLM genera lenguaje. No constituye por sí solo una base de datos ni reemplaza las fuentes de información.

¿Qué problema resuelve RAG?

Nuestros documentos no están necesariamente dentro del LLM

Supongamos que tenemos:

- manuales propios;
- procedimientos internos;
- documentación reciente;
- información privada;
- versiones específicas de dispositivos.

Necesitamos recuperar esa información antes de pedir una respuesta.



RAG permite proporcionar al modelo información recuperada desde nuestras propias fuentes.

Retrieval-Augmented Generation

Recuperación + generación



La recuperación y la generación son etapas diferentes.

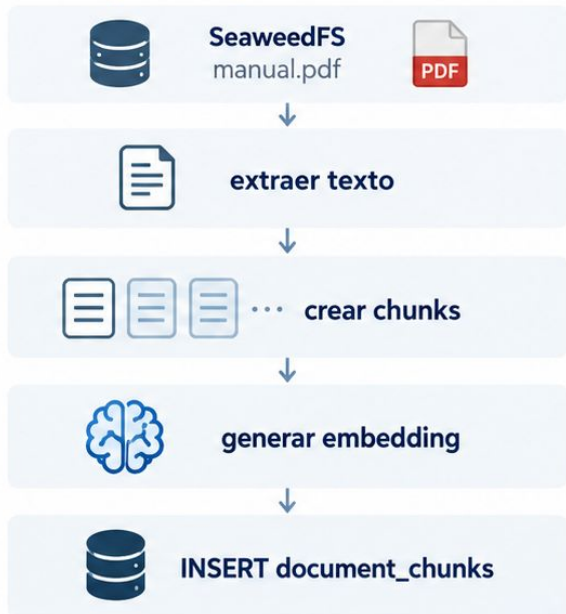


RAG conecta un mecanismo de recuperación con un modelo generativo.

Preparar la información

Del documento original al índice vectorial

El proceso puede pensarse como:



Pseudocódigo:

```
for chunk in chunks:
    vector = embedding_model(chunk.texto)

    guardar_chunk(
        documento_id,
        chunk.texto,
        vector
    )
```

Este proceso ocurre cuando incorporamos o actualizamos documentación.



Indexar significa preparar el contenido para que posteriormente pueda recuperarse de manera eficiente.

Buscar el contexto

La pregunta también se transforma en un vector

Pregunta:



“¿Cómo restablezco la conectividad del ENV-X?”



Pregunta

Texto en lenguaje natural



embedding_model

Convierte la pregunta en un vector



query_embedding

Vector numérico de alta dimensión

Luego consulta:

```
SELECT
  c.texto,
  c.pagina,
  d.nombre
FROM document_chunks c
JOIN documentos d
  ON d.documento_id = c.documento_id
ORDER BY c.embedding <=> :query_embedding
LIMIT 4;
```



Resultado:

Top 4 chunks relevantes



1

“Para restablecer la conectividad del ENV-X debe reiniciarse el servicio de red...”

p. 12 | Guía de operación



2

“El ENV-X utiliza la interfaz eth0 para la conexión de red... Verifique la configuración...”

p. 8 | Manual técnico



3

“Si no hay conectividad en el ENV-X, revise el estado del firewall y las reglas de acceso...”

p. 15 | Procedimientos



4

“La pérdida de conectividad del ENV-X puede deberse a un error en la configuración DNS...”

p. 21 | Solución de problemas



La búsqueda vectorial recupera contexto. Todavía no construye la respuesta final.

Del contexto a la respuesta

Recuperar y generar son etapas diferentes

Una vez recuperados los fragmentos:

```
contexto = "\n\n".join(chunks)
prompt = f"""
Respondé utilizando el contexto
proporcionado.
Contexto:
{contexto}
Pregunta:
{pregunta}
"""
```

```
respuesta = llm(prompt)
```



El modelo de embeddings ayuda a encontrar información. El LLM utiliza esa información para redactar una respuesta.

¿De dónde salió la respuesta?

Conservar la relación con la fuente

Cada fragmento puede conservar:

- documento_id
- version
- pagina
- seccion
- object_key

Además, si cambia el documento, debemos considerar la actualización de sus chunks y embeddings.

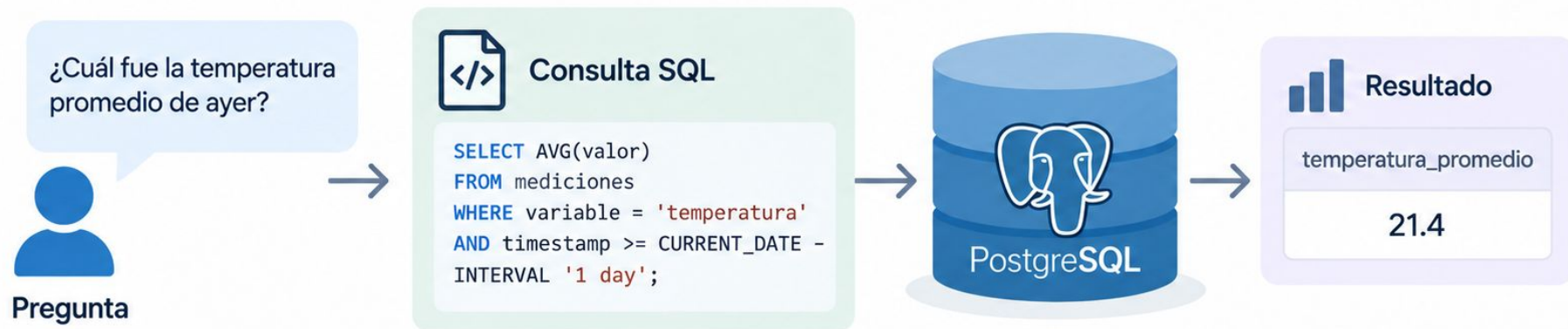


Una arquitectura RAG necesita administrar no solamente vectores, sino también origen, versiones y ciclo de vida.

9

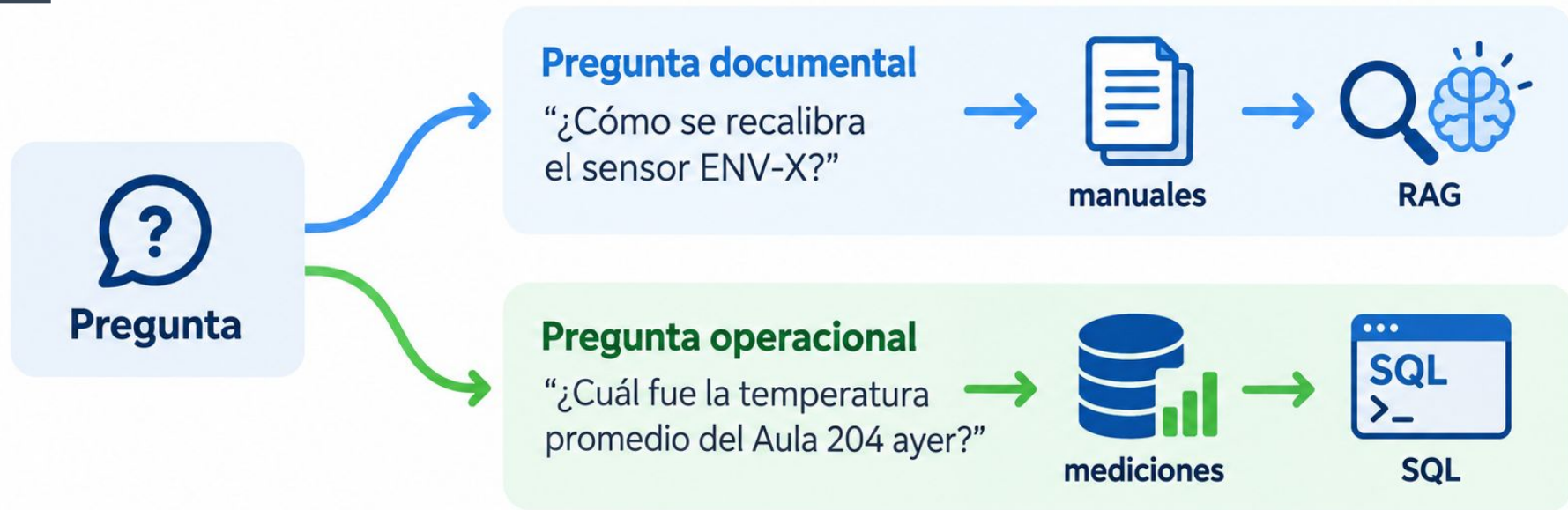
Text-to-SQL

Utilizar lenguaje natural para consultar datos estructurados.



No todo está en los documentos

Algunas preguntas necesitan consultar la base



La fuente de información determina qué mecanismo de recuperación necesitamos.

Text-to-SQL

El LLM puede construir una consulta

El modelo necesita recibir información sobre el esquema y la pregunta.

Ejemplo de contexto:

Tabla ubicaciones:

- ubicacion_id
- nombre

Tabla dispositivos:

- ubspositivo_id
- ubicacion_id

Tabla mediciones:

- ubspositivo_id
- variable
- valor
- timestamp

Pregunta:

“¿Cuál fue la temperatura promedio del Aula 204 durante las últimas 24 horas?”



**Esquema +
pregunta**



LLM



SQL



Text-to-SQL necesita comprender la estructura disponible antes de poder construir una consulta razonable.

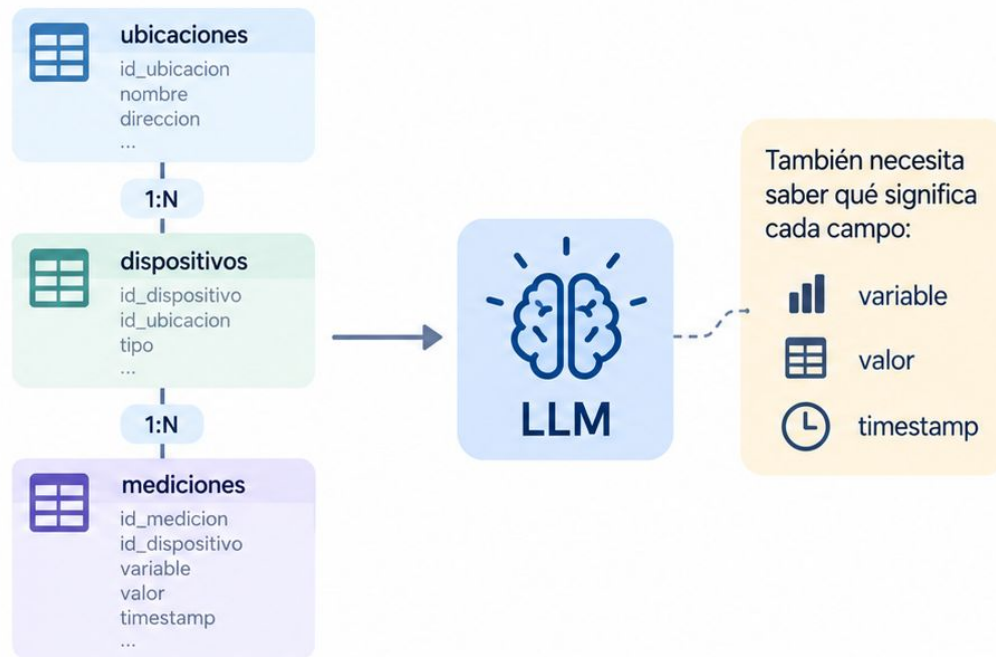
La IA necesita contexto estructural

Tablas, columnas y relaciones

No alcanza con decir:

“Tenemos mediciones.”

El modelo necesita comprender relaciones como:



La generación de SQL depende de esa información.



Un buen modelo de datos también facilita que otras herramientas puedan interpretar correctamente el esquema.

El SQL generado

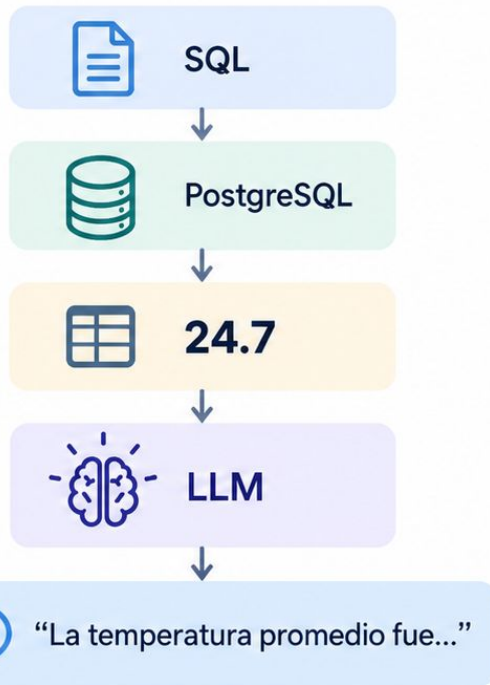
La pregunta se transforma en operaciones sobre el modelo

Pregunta:

“¿Cuál fue la temperatura promedio del Aula 204 durante las últimas 24 horas?”

Una consulta posible:

```
SELECT AVG(m.valor)
FROM mediciones m
JOIN dispositivos d
  ON d.dispositivo_id =
     m.dispositivo_id
JOIN ubicaciones u
  ON u.ubicacion_id =
     d.ubicacion_id
WHERE u.nombre = 'Aula 204'
AND m.variable = 'temperatura'
AND m.timestamp >=
     NOW() - INTERVAL '24 hours';
```



El LLM construye la consulta. PostgreSQL continúa siendo quien consulta y calcula sobre los datos almacenados.

Controlar el acceso

El modelo no debería utilizar un usuario administrador

Podemos crear un usuario específico:

```
CREATE ROLE ai_readonly  
LOGIN PASSWORD '*****';
```

Y permitir solamente consultas:

```
GRANT SELECT  
ON dispositivos,  
ubicaciones,  
mediciones  
TO ai_readonly;
```



Todavía quedan otras decisiones:

- qué tablas puede consultar;
- cuántas filas puede recuperar;
- cuánto puede tardar;
- qué datos pertenecen a cada usuario u organización.

Estos temas serán parte de la próxima clase.

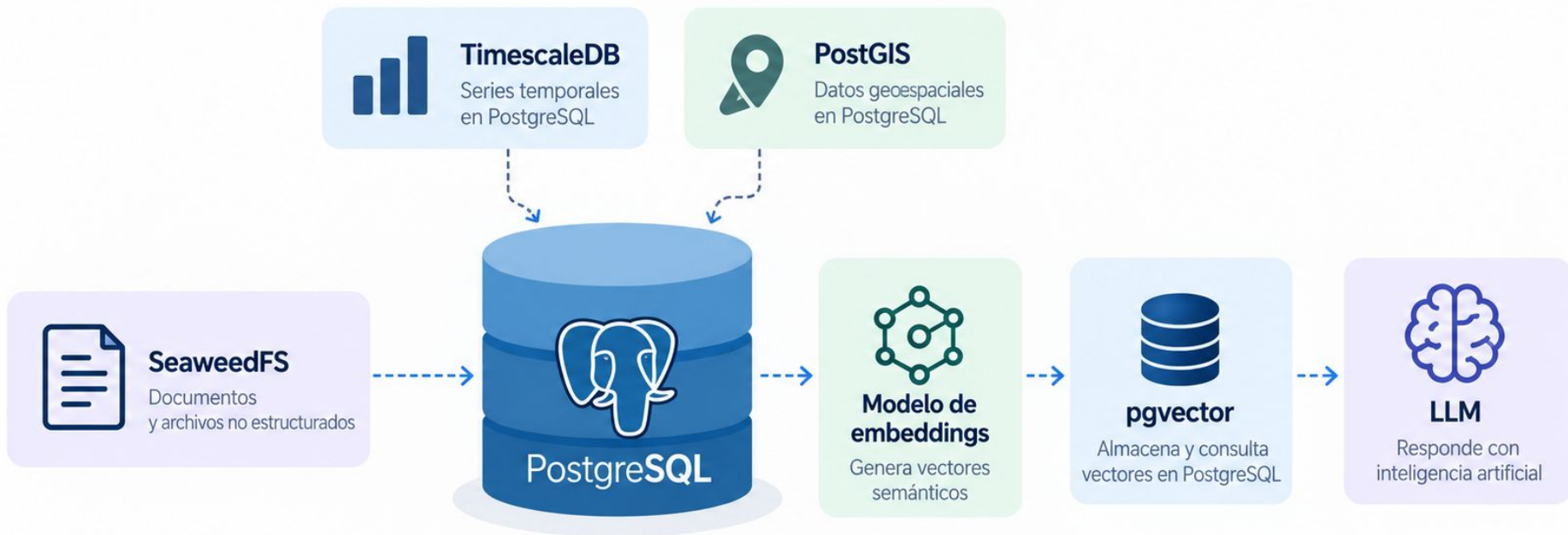


Automatizar consultas no elimina los controles de seguridad: vuelve todavía más importante definir permisos adecuados.

10

Práctica Integradora

Combinar datos estructurados, documentos, almacenamiento especializado e inteligencia artificial.



Datos estructurados

Series temporales, geolocalización y consultas en PostgreSQL

Partimos de una solución con:

- **PostgreSQL** como base principal;
- **TimescaleDB** para organizar mediciones temporales;
- **PostGIS** para representar ubicación de dispositivos.

Tenemos:

- dispositivos;
- ubicaciones;
- mediciones;
- coordenadas.

Debemos:

1. registrar dispositivos y sus ubicaciones;
2. almacenar mediciones temporales;
3. organizar la tabla de mediciones como hypertable;
4. representar geográficamente la ubicación;
5. consultar históricos recientes;
6. calcular agregaciones temporales;
7. realizar consultas espaciales de proximidad.

Ejemplos de preguntas:

"¿Cuál fue la temperatura promedio del Aula 204 durante las últimas 24 horas?"

"¿Qué dispositivos se encuentran a menos de 2 km de esta ubicación?"



La práctica debe mostrar que el mismo núcleo relacional puede especializarse para resolver consultas temporales y espaciales.

Recuperación documental

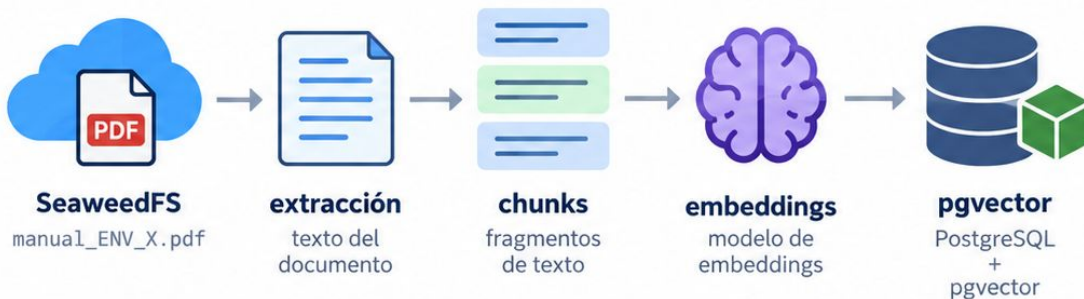
SeaweedFS, pgvector y embeddings

Partimos de:

manual_ENV_X.pdf
almacenado en SeaweedFS.

Debemos:

1. registrar sus metadatos en PostgreSQL;
2. extraer el texto;
3. dividirlo en chunks;
4. conservar documento, página y sección;
5. generar embeddings;
6. almacenarlos mediante pgvector;
7. realizar una búsqueda semántica.



Pregunta de prueba:

“¿Cómo debe recalibrarse el sensor ENV-X después de reemplazar la batería?”



pregunta

búsqueda semántica

	0.92
	0.87
	0.83

resultados
top fragments



La práctica debe permitir seguir el recorrido completo desde el archivo original hasta los fragmentos recuperados semánticamente.

Orquestar múltiples fuentes

Text-to-SQL para datos estructurados y RAG para documentos

Pregunta 1:

¿Cuál fue la temperatura promedio del Aula 204 durante las últimas 24 horas?



Pregunta 2:

¿Cómo debe recalibrarse el sensor ENV-X?



Pregunta integradora:

El sensor ENV-X del Aula 204 presenta mediciones anómalas. ¿Cuáles fueron sus valores promedio durante las últimas 24 horas y qué procedimiento de calibración indica su manual?



Una única respuesta puede requerir consultar datos estructurados, datos temporales, ubicación y documentación técnica.

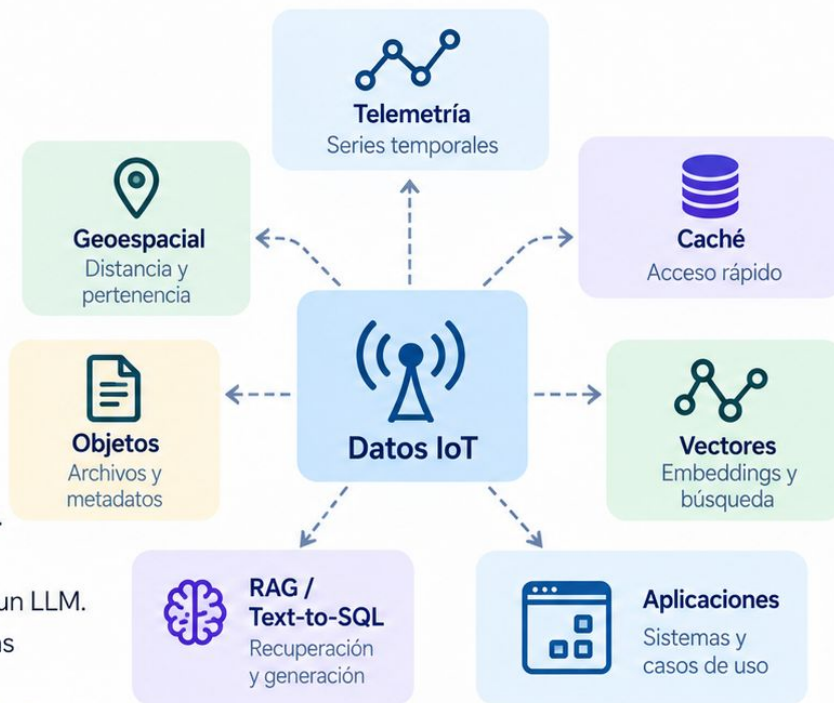


Para ir cerrando..

Resumen de la clase

Ideas principales

- Los datos IoT pueden presentar necesidades de almacenamiento diferentes.
- Las series temporales organizan observaciones alrededor del tiempo.
- Agregaciones, retención y downsampling permiten gestionar históricos crecientes.
- La información geoespacial incorpora relaciones de distancia y pertenencia.
- La caché resulta útil para información temporal o consultada frecuentemente.
- Object Storage permite separar archivos de sus metadatos.
- Los documentos pueden transformarse en chunks.
- Los modelos de embeddings representan contenido mediante vectores.
- La búsqueda vectorial permite recuperar contenido por similitud.
- RAG combina recuperación de información con generación mediante un LLM.
- Text-to-SQL permite traducir preguntas en lenguaje natural a consultas estructuradas.
- Diferentes mecanismos pueden coexistir dentro de una misma arquitectura.



La arquitectura de datos determina qué información conservamos, cómo la recuperamos y de qué manera puede ser utilizada posteriormente por aplicaciones y sistemas de IA.

Próxima clase

Administración, Seguridad y Escalabilidad

Hasta ahora nos concentramos principalmente en cómo representar y almacenar la información.

En la próxima clase vamos a analizar:

- usuarios;
- autenticación;
- roles y permisos;
- principio de mínimo privilegio;
- aplicaciones multi-tenant;
- aislamiento de datos;
- Row Level Security;
- transacciones y concurrencia;
- bloqueos;
- backup y recuperación;
- replicación;
- particionamiento;
- escalabilidad.



Hoy aprendimos a combinar mecanismos de almacenamiento.
La próxima clase veremos cómo administrar, proteger y escalar esa arquitectura.

Evaluación 3

Clases 5 y 6



Las preguntas combinarán comprensión, interpretación y selección tecnológica.



El objetivo es reconocer qué mecanismo responde a cada necesidad y justificar la decisión.



Enlace a la encuesta



Muchas gracias ¿Dudas?